# U.E 5.1 - Intelligence Artificielle : Machine learning
# Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France

BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

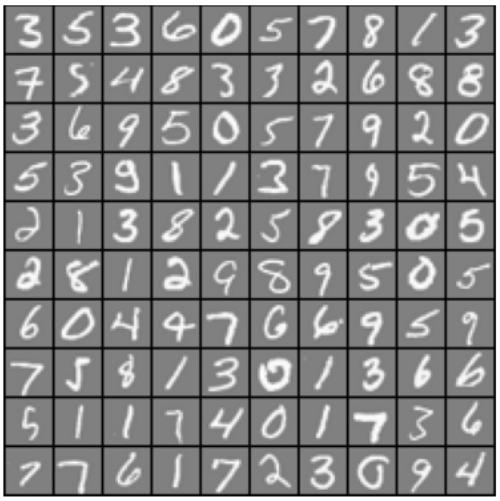
## Sommaire

**1 Visualizing the data** **2**
1.1 Prediction by a neural network . . . 2
1.2 Model representation . . . 2
1.3 Feedforward Propagation and Prediction . . . 2

**2 Learning a classifier with a neural Network** **3**
2.1 Feedforward and cost function . . . 3
2.2 Regularized cost function . . . 4
2.3 Backpropagation . . . 4
2.3.1 Sigmoid gradient . . . 4
2.3.2 Random initialization . . . 5
2.3.3 The backpropagation algorithm . . . 6
2.3.4 Gradient checking . . . 6
2.3.5 Regularized Neural Networks . . . 6
2.3.6 Learning parameters using fmin_cg . . . 6
2.4 Visualising the hidden layer . . . 7
2.5 Final analysis . . . 7

**3 Some questions** **8**

# 1 Visualizing the data



## 1.1 Prediction by a neural network

## 1.2 Model representation

## 1.3 Feedforward Propagation and Prediction

```python
import numpy as np

from sigmoid import sigmoid

def predictNeuralNetwork(Theta1, Theta2, X):
    """ outputs the predicted label of X given the
    trained weights of a neural network (Theta1, Theta2)
    """

# Useful values
    m, _ = X.shape
    num_labels, _ = Theta2.shape

# ====================== YOUR CODE HERE ====================

    a1 = X

    z2 = a1 @ Theta1.T
    a2 = sigmoid(z2)
    a2 =  np.column_stack((np.ones((m,1)), a2))

    z3 = a2 @ Theta2.T
```

```
    a3 = sigmoid(z3)

    p = np.argmax(a3, axis=1)

# =========================================================

    return p
```

# 2 Learning a classifier with a neural Network

## 2.1 Feedforward and cost function

```
import numpy as np

from sigmoid import sigmoid
from sigmoidGradient import sigmoidGradient


def nnCostFunction(nn_params, input_layer_size,
   hidden_layer_size, num_labels, X, y, Lambda):

    theta1 = nn_params[0:(hidden_layer_size*(input_layer_size
       +1))].reshape((input_layer_size+1),hidden_layer_size).T
    theta2 = nn_params[(hidden_layer_size*(input_layer_size+1)
       ):].reshape((hidden_layer_size+1),num_labels).T

# Setup some useful variables
    m, _ = X.shape

# You need to return the following variables correctly
    X = np.column_stack((np.ones((m, 1)), X))
    z2 = X.dot(theta1.T)
    a2 = sigmoid(z2)
    a2 = np.column_stack((np.ones((m, 1)), a2))
    z3 = a2 @ theta2.T
    a3 = sigmoid(z3)
    # h = np.argmax(a3, axis=1)
    y_train = np.zeros((m, num_labels))
    y_train[np.arange(m), y.flatten()] = 1

    J = 1/m * np.sum(-y_train*np.log(a3) - (1-y_train)*np.log
       (1-a3))
```

- Output:

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex03_startercode$ python3 ex3_nn_train.py
 ------------------------

Loading Saved Neural Network Parameters ...

 ------------------------

Feedforward Using Neural Network ...

 ------------------------

Cost at parameters (loaded from ex3weights): 0.287629
(this value should be about 0.287629)
```

## 2.2 Regularized cost function

```
    ...
    J = 1/m * np.sum(-y_train*np.log(a3) - (1-y_train)*np.log
       (1-a3))

     # Cost regularisation
    reg = Lambda/(2*m) * (np.sum(theta1[:,1:]**2) + np.sum(
       theta2[:,1:]**2))
    J = J + reg
```

- Output:

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex03_startercode$ python3 ex3_nn_train.py
Cost at parameters (loaded from ex3weights): 0.287629
(this value should be about 0.287629)


 ------------------------

Checking Cost Function (w/ Regularization) ...

 ------------------------

Cost at parameters (loaded from ex3weights): 0.383770
(this value should be about 0.383770)

 ------------------------
```

## 2.3 Backpropagation

### 2.3.1 Sigmoid gradient

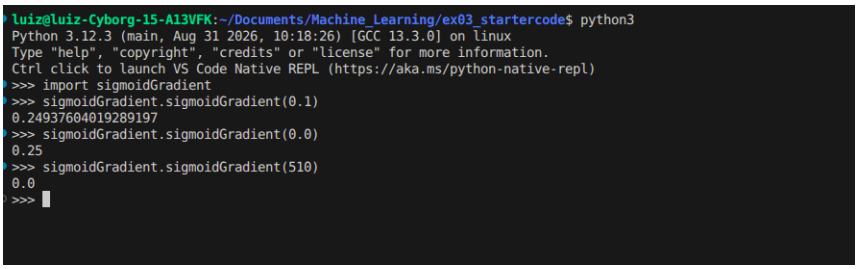
```
from sigmoid import sigmoid

def sigmoidGradient(z):
    """computes the gradient of the sigmoid function
    evaluated at z. This should work regardless if z is a
       matrix or a
    vector. In particular, if z is a vector or matrix, you
       should return
    the gradient for each element."""

# ===================== YOUR CODE HERE ======================
# Instructions: Compute the gradient of the sigmoid function
   evaluated at
#               each value of z (z can be a matrix, vector or
   scalar).
```

```
12
13      g = sigmoid(z) * (1 - sigmoid(z))
14
15  #
    ============================================================
16
17      return g
```

- Output:




### 2.3.2 Random initialization

```
1   import numpy as np
2
3   def randInitializeWeights(L_in, L_out):
4       """randomly initializes the weights of a layer with L_in
           incoming connections and L_out outgoing
5         connections.
6
7         Note that W should be set to a matrix of size(L_out, 1 +
              L_in) as the 1st column of W handles the "bias"
            terms
8       """
9
10      # ======================== YOUR CODE HERE
           ==================================
11      # Instructions: Initialize W randomly so that we break the
            symmetry while
12      #               training the neural network.
13      #
14      # Note: The first row of W corresponds to the parameters
           for the bias units
15      #
16
17      epsilon_init = 0.12
18      W = np.random.rand(L_out, 1+L_in) * 2 * epsilon_init -
           epsilon_init
19
```

```
20
21      # =========================================================
22
23      return W
```

### 2.3.3 The backpropagation algorithm

### 2.3.4 Gradient checking

- Output:

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex03_startercode$ python3 ex3_nn_train.py
 [ 1.64567932e-01  1.64567932e-01]
 [ 5.77867378e-02  5.77867378e-02]
 [ 5.07530173e-02  5.07530173e-02]
 [ 1.58339334e-01  1.58339334e-01]
 [ 5.59235296e-02  5.59235296e-02]
 [ 4.91620841e-02  4.91620841e-02]
 [ 1.51127527e-01  1.51127527e-01]
 [ 5.36967009e-02  5.36967009e-02]
 [ 4.71456249e-02  4.71456249e-02]
 [ 1.49568335e-01  1.49568335e-01]
 [ 5.31542052e-02  5.31542052e-02]
 [ 4.65597186e-02  4.65597186e-02]]
The above two columns you get should be very similar.
(Left-Your Numerical Gradient, Right-Analytical Gradient)


If your backpropagation implementation is correct, then
 the relative difference will be small (less than 1e-9).

Relative Difference: 2.40441e-11
```

The relative difference between the analytical and numerical gradients is very small, which indicates that the backpropagation implementation is correct.

### 2.3.5 Regularized Neural Networks

### 2.3.6 Learning parameters using fmin_cg

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex03_startercode$ python3 ex3_nn_train.py

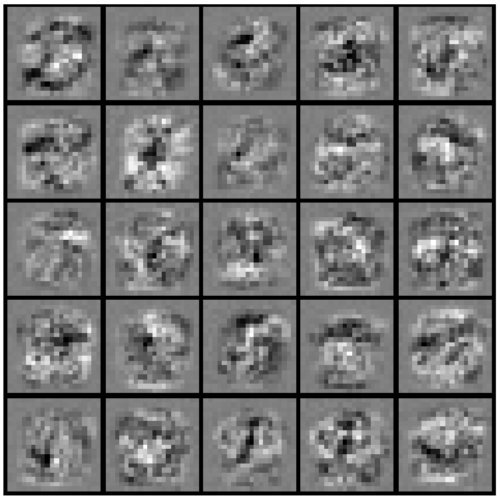
 -------------------------

Training Neural Network...
/usr/lib/python3/dist-packages/scipy/optimize/_optimize.py:1698: OptimizeWarning: Maximum number of iterations has been exceeded.
  res =  minimize_cg(f, x0, args, fprime, callback=callback, **opts)
         Current function value: 0.363290
         Iterations: 100
         Function evaluations: 225
         Gradient evaluations: 225

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 98.420000
```

## 2.4 Visualising the hidden layer



## 2.5 Final analysis

- Results if $\lambda = 0.1$ and $maxIter = 150$:

```
luiz@luiz-Cyborg-15-A13VFK:~/Documents/Machine_Learning/ex03_startercode$ python3 ex3_nn_train.py

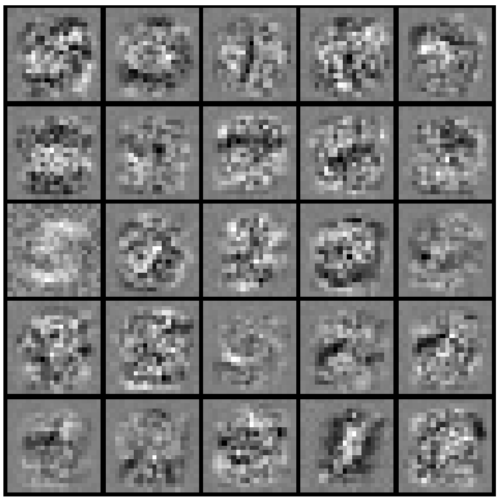
 -------------------------

Training Neural Network...
/usr/lib/python3/dist-packages/scipy/optimize/_optimize.py:1698: OptimizeWarning: Maximum number of iterations has been exceeded.
  res = _minimize_cg(f, x0, args, fprime, callback=callback, **opts)
        Current function value: 0.091292
        Iterations: 150
        Function evaluations: 380
        Gradient evaluations: 380

 -------------------------

Visualizing Neural Network...
Training Set Accuracy: 99.840000
```

- Displaying the hidden layers:



# 3 Some questions

- Décrivez l'approche basée sur les réseaux de neurones. Quelle modélisation est retenue? Quels sont les paramètres à apprendre?

  The neural network is an approach that allows us to solve complex real world problems. By adding hidden layers, we can build non-linear decision boundaries of various shapes. The parameters to learn are the weights connecting the neurons between each layer, usually defined as Theta matrices.

- Quelle est la fonction de coût? Quelles modifications sont effectuées par rapport à celle utilisée en TP2?

  The cost function is a mathematical metric that measures the difference between the computed predictions and the actual expected values. It helps us track if the network is training correctly, because if the cost grows or stays constant after each iteration, something is wrong. Here, we are dealing with a multi-class problem, so the cost function uses a double sum to compute the error across all output classes for each example. In contrast, TP2 was a binary classification problem, so we only needed a single sum.